

COMP 356
Programming Language Structures
Notes for Chapter 9 of *Concepts of Programming Languages*
Subprograms

Subprograms implement **process abstraction** - the ability to define and then use a computation without concern for how it is implemented.

Definitions:

- a **function** is a subprogram that has a return value. A function call is an expression, so functions extend the expressions of the programming language.
- a **procedure** is a subprogram that has no direct return value. Procedures are executed for their side-effects (output, changes to parameters or global variables, ...) and so extend the statements of the programming language.
- the parameters in a subprogram definition are called **formal parameters** (or **formals**)
- the parameters in a subprogram call are called **actual parameters** (or **actuals**)
- several kinds of correspondence between formal and actual parameters are possible:
 - with **positional parameters**, the correspondence between formal and actual parameters is based only on position (the first actual is bound to the first formal, ...)
 - with **keyword parameters**, the programmer specifies which formal the each actual is bound to by name in the subprogram call. The programmer does not need to remember the order of the formal parameters, but must remember their names.
 - with **default parameters**, some parameters have default values specified in the subprogram definition, and those parameters can be omitted in calls to the subprogram. For example, the following C++ function:

```
double log(double x, int base = 10) { ... }
```

can be called with only one argument, in which case the value of formal parameter **base** will be 10.

In languages with default parameters, one of the following designs is usually used:

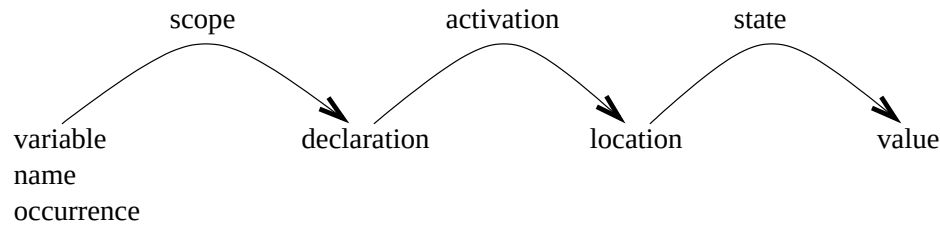
- * (C++) all default parameters must come last in parameter lists, and once an actual parameter is omitted in a call, all following parameters must also be omitted.
- * (Visual Basic) parameters are omitted by using commas with no actual parameters between them.
- * (Ada) keyword parameters are used in conjunction with default parameters so that any needed parameters can be supplied by name.

Note that functions with default parameters in C++ are different from unchecked functions.

- a **subprogram declaration** is a declaration of the name, parameter types and return type (if the subprogram is a function) of a subprogram, but with no definition (subprogram body). Subprogram declarations:
 - are called **function prototypes** in C and C++
 - are called **forward declarations** in Pascal and Ada
 - allow calls to the subprogram to be compiled (and typechecked) before the subprogram has been defined. This is useful for mutually recursive subprograms and for separate compilation.
 - are not needed in Java and C#, because a method can be called before a declaration (or definition) of that method has been seen in these languages

- a subprogram is **active** if it is executing, i.e. it has been called and has not yet returned. An execution of a subprogram is also called an **activation** of that subprogram.
- a subprogram is **recursive** if it calls itself (directly or indirectly). Equivalently, a subprogram is recursive if it can have multiple simultaneous activations.
- a **referencing environment** is a collection of all names that are visible at a particular point in the execution of a program. Since subprograms can have parameters and local variables, each subprogram introduces a new referencing environment.

The following diagram shows the role of an activation in mapping an occurrence of a stack-dynamic variable to its value.



1 Parameter Passing Methods

Parameter passing methods differ on whether parameters are used to transmit data to the subprogram (**in mode** parameters), from the subprogram back to the caller (**out mode** parameters) or both (**inout mode** parameters).

Parameter passing can be implemented in several ways, including:

- pass-by-value
- pass-by-reference
- pass-by-value-result
- pass-by-name
- macro expansion

Pass-by-value implements in mode parameters, while the others implement some form of inout mode parameters.

1.1 Pass-by-Value

Under pass-by-value, only the value of the parameter is passed, and this value is placed in storage local to the subprogram. Hence, any modifications made to the formal parameter are not visible to the caller.

For the prototype: $P(x)$ (types omitted) and call: $P(E)$, a subprogram call with pass-by-value parameters is executed as:

```

x = E;
body of P
return result if necessary

```

For example, in the following code sequence:

```

void swap(int x, int y) {
    int t;

    t = x;
    x = y;
    y = t;
}

```

...

```

int a = 3, b = 4;
swap(a, b);

```

the call to `swap` under pass-by-value is executed as:

```

x = a;
y = b;
t = x;
x = y;
y = t;

```

so the values of `a` and `b` are unchanged.

Value parameters in Pascal and all parameters in C, C++ and Java are pass-by-value parameters. However, note that passing an address (or reference) by value allows the subprogram to change the referred-to memory location. Pass-by-value is the default in C#.

1.2 Pass-by-Reference

Under pass-by-reference, the address of the actual parameter (or some other **access path** to the actual) is passed. Hence, only l-values can be passed by-reference, and any modifications made to the formal parameter are visible to the caller.

For the prototype: `P(x)` and call: `P(E)`, a subprogram call with pass-by-reference parameters is executed as:

```

x = &E;
body of P
return result if necessary

```

The previous `swap` example under pass-by-reference is executed as:

```

x = &a;
y = &b;
t = *x;
*x = *y;
*y = t;

```

so the values of `a` and `b` are swapped.

Pass-by-reference can give surprising results in the presence of aliasing. For example:

```

void foo(int x, int y, int arr2[]) {
    x = 2;
    arr2[y] = 42;
}

```

...

```

int a = 0;

```

```
int arr[] = {0, 1, 2, 3, 4, 5};

foo(a, a, arr);
```

Under pass-by-reference, this call aliases **x** and **y** – both refer to the location of **a**. Hence, after this call **a** is 2 and **arr[2]** is 42, not **arr[0]**. Note that passing a global variable as a reference parameter also creates an alias.

Pascal **var** parameters and C# **ref** parameters are passed by-reference, and passing a pointer or reference in C, C++ and Java gives the effect of passing the referred-to location by reference. When C++ **const** pointers and references and Java **final** references are used as formal parameters, the implicit variable referred to by the pointer or reference can not be assigned to in the body of the subprogram. This allows a limited form of pass-by-value without the overhead of copying. Python and Ruby use pass-by-assignment, which is similar to passing references in Java.

1.3 Pass-by-Value-Result

Under pass-by-value-result (also called **copy-in, copy-out**), constants and expression results are passed by-value. L-values are passed as follows:

1. copy-in phase: the values and addresses of the actuals are computed and stored. The values are stored in the formals, while the addresses are placed in other temporary storage.
2. the body of the subprogram is executed
3. copy-out phase: the values of the formals are copied out into the actuals using the saved addresses

Hence, any modifications made to the formal parameter are visible to the caller.

The previous **swap** example under pass-by-value-result is executed as (using **px** and **py** as the storage for the addresses):

```
// copy-in phase
x = a;
px = &a;
y = b;
py = &b;

// executing body
t = x;
x = y;
y = t;

// copy-out phase
*px = x;
*py = y;
```

so the values of **a** and **b** are swapped.

The example using function **foo** is executed as follows under pass-by-value-result:

```
// copy-in phase
x = a;
px = &a;
y = a;
py = &a;
copy(arr2, arr); // copy the contents of arr to arr2
parr2 = &arr;

// executing body
```

```

x = 2;
arr2[y] = 42;

// copy-out phase
*px = x;
*py = y;
copy(*parr2, arr2);

```

Results:

- `arr[0]` is 42 and `arr[2]` is unchanged
- the final value of `a` depends on the order that the copy-outs occur in – it could be either 0 or 2. This is usually implementation dependent in languages that use pass-by-value-result.
- in the presence of aliasing, pass-by-reference and pass-by-value-result can exhibit different behaviors.

Another example:

```

int g = 5;

void foo(int x) {
    x = 3;
    g = 4;
}

...

foo(g); // aliases g and x under pass-by-reference

```

Under pass-by-reference, the value of `g` after the call to `foo` is 4. Under pass-by-value-result, the value of `g` after the call to `foo` is 3 (because of the copy-out).

Ada has three parameter passing modes:

- **in** parameters (by value)
- **out** parameters (just copy-out – also called **pass-by-result** parameters because they only pass information from the subprogram to the caller). C# also has out parameters.
- **in out** parameters (by value-result or by reference)

In Ada 83, the use of value-result or reference for in out parameters is implementation dependent. Any program that gives a different result depending on the implementation of parameter passing is defined to be illegal. However, in general compilers can not detect all aliasing, so Ada 83 programs are not portable. (!)

The Ada 95 standard specifies that in out parameters of scalar types (numbers, booleans, ...) are passed by-value-result, and in out parameters of structured types (arrays, records, ...) are passed by-reference. This removes the portability problem, and also avoids making copies of large arrays and records when they are passed as in out parameters. However, Ada 95 programmers must remember two sets of rules for in out parameters.

1.4 Macros

Many programming languages (C, C++, Scheme, ...) include **macros**, which are similar to subprograms but avoid the overhead of subprogram call and return. A call to a macro is executed as follows:

1. the actual parameters are textually substituted for the formals
2. the resulting macro body is textually substituted for the call

so no precautions to avoid capture are taken. A variable is **captured** during a substitution if it is bound to a different declaration than it was before the substitution. For example, the C macro:

```
#define addx(y) (x + y)
```

appears to refer to a global `x` (under static scope), but this `x` will be captured by any local declaration of `x` in the scope of the call, i.e.:

```
#include <iostream>
int x = 5;
#define addx(y) (x + y)

int main() {
    int x = 2;
    std::cout << addx(6) << std::endl;
}
```

prints 8, not 11. In general, macro expansion gives the effect of dynamic scope.

In C and C++, the preprocessor expands macros so that the compiler never sees them.

Macros often give surprising results. For example:

```
#include <iostream>
#define swap(x, y) ({int t = x; x = y; y = t;})

int main() {
    int i = 2, a[] = {1, 2, 3, 4};
    swap(i, a[i]);
    // here i == 3 and a == {1, 2, 3, 2}
```

Pass-by-name gives the same result on this example, but pass-by-reference and pass-by-value-result give `a == {1, 2, 2, 4}`.

Another example:

```
#include <iostream>
#define prod(x, y) (x * y)

int main() {
    std::cout << prod(2, 3 + 4) << std::endl;
}
```

prints 10 (not 14).

Fix:

```
#define prod(x, y) ((x) * (y))
```

Another example:

```
#define fact(n) ((n) > 0? (n) * fact((n) - 1) : 1)
```

This blows up the preprocessor of a traditional (Kernigan & Ritchie) C compiler. Under ANSI C and C++, no replacement of a “recursive” macro call is done, so this macro has an undefined function call in its body.

1.5 Multidimensional Arrays as Parameters

In C and C++:

- to permit index calculations (under row-major layout), all array dimensions except the first must be included in the formal parameter. For example:

```
void spindleArray(int arr[][10][5]) {
    ...
}
```

- this design does not allow one function to manipulate arrays of different sizes
- alternative: pass a pointer to the array and the relevant dimensions of the array. For example:

```
void spindleArray(int *arr, int dim2, int dim3) {
    ...
}
```

This requires the programmer to perform all address calculations (using pointer arithmetic) in the body of `spindleArray`.

In Ada, Java and C#:

- multidimensional arrays are actually one-dimensional arrays of arrays
- arrays include their size as an attribute (or instance variable)
- hence, array index calculations can always be done, so multidimensional array parameters need not specify any of the dimensions. Example (Java):

```
void spindleArray(int[] [] [] arr) {
    ...
}
```

2 Other Aspects of Subprograms

2.1 Subprograms as Parameters

It is often useful to pass a subprogram as a parameter to another subprogram. For example, Scheme has a `map` function that takes a function and a list as parameters, and returns the result of applying the function to every element of the list.

Some design issues:

- Can subprograms be passed as parameters?
- Are subprogram parameters typechecked?
- What referencing environment are calls to subprogram parameters evaluated in?

By programming language:

- Ada and Java do not allow subprograms as parameters.
- Early versions of Pascal allowed subprogram parameters but did not typecheck them (when passed or when applied).
- Later versions of Pascal require parameter and return types for subprogram parameters to be given, and so typecheck such parameters.
- C and C++ allow pointers to functions as parameters. Function pointer types include parameter and return types, so such parameters are typechecked.

In languages that use static scope, subprogram parameters are evaluated in the referencing environment that the actual parameter was defined in. This is always the global environment in C and C++, but is interesting in Pascal and Ada, where function definitions can be nested. Evaluating subprogram parameters in the environment they are defined in is called **deep binding**.

In languages that use dynamic scope (SNOBOL), subprogram parameters are evaluated in the referencing environment of the call to the subprogram parameter. This is called **shallow binding**.

2.2 Overloading

A subprogram name is **overloaded** if it is bound to multiple subprogram definitions. The compiler must be able to determine (statically) which definition is meant in a particular call by the number and type of the actual parameters, and possibly by the return type.

In Ada, the number and type of parameters and the return type (for functions) are used to resolve overloading. Because Ada does not allow mixed-mode expressions, the return type of the function can be inferred from the context of the call.

In C++, Java and C#, only the number and type of parameters are used to resolve overloading. Both languages allow mixed-mode expressions and also allow functions to be called in statement contexts (where the return value is ignored), so the return type of the function can not be inferred from the context of the call. Overloading is used heavily in these languages, particularly for constructors.

It is illegal to omit default parameters that would make it impossible for the compiler to resolve overloading. For example, if a program had both of the following function definitions:

```
double log(double x, int base = 10) { ... }
double log(double x) { ... }
```

then the call:

```
log(2.3)
```

is illegal. Note that in this case, the second version of `log` is useless (it can't be called) and the default parameter can never be omitted.

Note that overloading can simulate default parameters:

```
double log(double x, int base) { ... }
double log(double x) {return log(x, 10);}
```

2.3 Generic Subprograms

A subprogram is **generic** (or **polymorphic**) if it can take parameters of different types on different activations. This can be implemented in several ways:

- in dynamically typed languages (Scheme, LISP, Prolog, JavaScript, ...), arguments of any type can be passed to any subprogram. However, type errors can result if the subprogram makes assumptions about the types of its parameters.
- in languages that allow overloading (C++, Java, Ada, ...), an overloaded subprogram name can be viewed as polymorphic. This form of polymorphism is called **ad hoc** polymorphism.
- in languages with **parametric** polymorphism (C++, Ada, Java 5.0, ...), subprograms can have types as parameters. Typically, the type parameters must be used first to **instantiate** the subprogram, and then it can be called with the remaining parameters.

An example of defining and using a C++ template function:

```
#include <iostream>
#include <string>

template <class EType>
EType findMax(EType arr[], int len) {
    EType res = arr[0];

    for (int i = 1; i < len; i++) {
        if (arr[i] > res) res = arr[i];
    }
    return res;
}
```



```

}

int main() {
    std::string sarr[] = {"an", "array", "of", "strings"};
    int iarr[] = {5, 4, 3, 2, 1};

    std::cout << findMax<int>(iarr, 5) << std::endl;
    std::cout << findMax<std::string>(sarr, 4) << std::endl;
}

```

Macros are also polymorphic (and in fact early C++ compilers implemented template functions and classes using a form of macro expansion), but using template functions avoids the problems associated with macros.

Java 5.0 generic methods are more complex:

- generic parameters must be class types (not primitives)
- generic Java arrays can NOT be created, but generic Java array types are allowed as method parameters
- the method itself is not explicitly instantiated (when it is called), and in fact is instantiated only once
- restrictions can be specified on the range of classes that are passed as a parameter (i.e. that they must be subclasses of a particular class, or that they must implement a particular interface)
- the wildcard ? can be used to mean any Java class. The wildcard can also be restricted as in the previous point.

Example:

```

import java.util.*;

public class FindMax {

    // only collections of Integers can be passed to this method
    public static Integer findMax1(Collection<Integer> c) {
        Iterator<Integer> it = c.iterator();
        if (!it.hasNext()) {
            return null;
        } else {
            // can't use 5.0 style foreach loop here, because we don't want to
            // process the first element
            Integer i = it.next(); // no cast to Integer required
            Integer max = i;
            while (it.hasNext()) {
                i = it.next();
                if (i.intValue() > max.intValue()) max = i;
            }
            return max;
        }
    }

    // T is a type parameter that can be used throughout the method
    // the parameter declaration goes before the return type
    // the type name T can then be used throughout the method
    // the actual value of T is the base type of the array passed to the method
    public static <T extends Comparable> T findMax2(T [] c) {
        // pre: c.length > 0
    }
}

```

```

    T max = c[0];
    for (int i = 1; i < c.length; i++) {
        if (c[i].compareTo(max) > 0) max = c[i];
    }
    return max;
}

// using the ? wildcard - any Collection of any "subclass" of Comparable
// can be passed to this method
public static Comparable findMax3(Collection<? extends Comparable> c) {
    // pre: c.size() > 0
    Iterator<? extends Comparable> it = c.iterator();
    Comparable i = it.next(); // no cast to Comparable required
    Comparable max = i;
    while (it.hasNext()) {
        i = it.next();
        // the following line will fail (at runtime) if i and max are instances
        // of different classes
        if (i.compareTo(max) > 0) max = i;
    }
    return max;
}

public static void main(String argv[]) {
    ArrayList<Integer> ai = new ArrayList<Integer>();
    ai.add(new Integer(4));
    ai.add(new Integer(3));
    ai.add(new Integer(5));
    System.out.println(findMax1(ai));
    String [] sa = {"antelope", "zebra", "elephant", "rhino", "gazelle"};
    System.out.println(findMax2(sa));
    ArrayList<String> as = new ArrayList<String>();
    // ArrayList as = new ArrayList();
    as.add("antelope");
    as.add("zebra");
    as.add("elephant");
    // as.add(new Integer(3));
    System.out.println(findMax3(as));
}
}

```